

PRO

Programmation orientée objet

Leçon 2

Références nulles, valeurs par défaut, notion de package

Encapsulation

Exceptions et mot clé `static`

Référence nulle

On dit qu'une variable de type référence qui n'est pas liée à un objet est une référence nulle (mot réservé `null`).

L'appel d'une méthode sur une référence nulle ou la lecture d'un attribut nul déclenche une `NullPointerException` ce qui pour le moment arrête l'exécution.

```
Person p = null;

if(p == null) {
    System.out.println("p ne référence rien");
}

p.prettyPrint(); // NullPointerException !
```

Valeur par défaut des attributs

Contrairement aux variables locales (variables déclarées dans une méthode ou ses paramètres), les attributs d'un objet ont une valeur par défaut assignée lors de la création de l'objet, avant l'appel du constructeur.

Type	Default Value
boolean	false
byte	(byte) 0
short	(short) 0
int	0
long	0L
char	\u0000
float	0.0f
double	0.0d
object reference (including arrays)	null

Un package Java est un ensemble de types (classes, interfaces, énumérations, ...).

Par exemple, les classes fondamentales se trouvent dans `java.lang` (importé par défaut), les classes d'entrée/sortie se trouvent dans `java.io`, ...

A la première ligne d'un fichier, il faut préciser le package concerné (sinon il sera dans le package par défaut, ce qui n'est pas recommandé) :

```
package codeslides02.labo02;  
  
import java.util.Scanner;  
  
public class Program {  
    //...  
}
```

Package Java

Pour utiliser des types d'un package dans une classe d'un autre package :

soit on importe tous les types d'un package au début du fichier :

```
import labo01.*;
```

soit on importe un type précis contenu dans un package au début du fichier :

```
import labo01.Date;
```

soit on utilise le nom complet dans le code (déconseillé sauf si ambiguïtés) :

```
labo01.Date date = new labo01.Date(12, 11, 2023);
```

Si des classes se trouvent dans un même package, elles sont visibles sans importation.

L'encapsulation

Encapsulation

En Java, les attributs et méthodes peuvent être :

- `public` : accessibles de partout (intérieur, extérieur)

- `private` : accessibles seulement à l'intérieur de la classe

- `private-package/default` (par défaut si on ne précise rien) : accessibles à l'intérieur de la classe et à l'intérieur d'un package.

- `protected` : concerne l'héritage, voir plus loin

L'**encapsulation** est un concept OO **fondamental** qui consiste à **protéger l'information** contenue dans un objet.

Une classe doit être le vigile de l'intégrité de ses attributs.

Pour ce faire, on va généralement déclarer tous ses **attributs** en **private** et gérer leur intégrité via des **accesseurs**.

Attention, il ne faut faire des accesseurs que si c'est nécessaire.

Encapsulation : Accesseurs

Un **accesseur** est une méthode pour accéder, en lecture ou en écriture, aux attributs d'une classe.

Un **accesseur en lecture** (parfois appelé « *getteur* ») renverra en général la valeur de l'attribut. Son nom commence par `get` et est suivi du nom de l'attribut (propriété)

Un **accesseur en écriture** (parfois appelé mutateur ou « *setteur* ») est généralement `void`, prend un paramètre et modifie l'attribut, après avoir vérifié que cette modification ne portait pas atteinte à l'intégrité de l'objet. Son nom commence par `set`.

```
public class Person {  
    private double salary;  
  
    public double getSalary() {  
        return salary;  
    }  
  
    public void setSalary(double salary) {  
        if(salary >= 0) {  
            this.salary = salary;  
        }  
    }  
}
```

Niveaux de protection d'une classe

Nous avons vu que les membres d'une classe (attributs, constructeurs, méthodes, ...) peuvent avoir un niveau d'accès.

Une classe « top level », c'est à dire non imbriquée dans une classe (voir plus loin) peut aussi avoir un niveau d'accès parmi :

`public` : la classe peut être vue par tout le monde

private-package (si rien n'est précisé) : la classe n'est vue qu'à l'intérieur de son package

Il ne peut y avoir qu'une seule classe publique par fichier `.java` (car le nom du fichier `.java` doit être le nom de son unique classe publique).

Le mot clé `static`

Mot clé **static**

Un attribut ou une méthode **statique** se réfère à une **classe** et non à une instance particulière. On les appelle (de *préférence*) sur le nom de la classe et non sur une référence à un objet.

```
public class Person {  
    private String name;  
    private static int personCount;  
  
    public Person(String name) {  
        this.name = name;  
        ++personCount;  
    }  
  
    public static int getPersonCount() {  
        return personCount;  
    }  
}
```

```
public class Program {  
    public static void main(String[] args) {  
        System.out.println(Person.getPersonCount()); // 0  
  
        Person p1 = new Person("Alain");  
        System.out.println(Person.getPersonCount()); // 1  
  
        Person p2 = new Person("Ivan");  
        System.out.println(Person.getPersonCount()); // 2  
    }  
}
```

Constatez également que la méthode `main` est statique car il s'agit du point d'entrée du programme et qu'aucun objet n'existe avant son appel.

Mot clé **static**

Comme une méthode **statique** se réfère à une classe et non à une instance particulière, on ne peut **pas** utiliser le mot clé **this** dans une méthode statique.

Voici quelques exemples :

```
public class Statics {  
    public static final int CONST = 5;  
    public int attribute;  
  
    public static void testStatics() {  
        double pi = Math.PI; //pas besoin d'instancier un objet de type Math  
        int res = Math.max(7, 2);  
        int constant = Statics.CONST; //ou juste CONST car on est dans le bon contexte  
  
        String s = this.toString(); //erreur à la compilation !  
        int a = attribute; //erreur à la compilation !  
    }  
  
    public static void main(String[] args) {  
        testStatics();  
    }  
}
```

Mot clé `static`

Dans une classe `Date` :

```
public static boolean isLeapYear(int year) {  
    return (year % 100 != 0 && year % 4 == 0) || (year % 400 == 0);  
}
```

Cette classe fournit donc un « outil » permettant de savoir si une année est bissextile sans devoir créer un objet de type `Date` :

```
if(Date.isLeapYear(2024)) {  
    System.out.println("OK");  
}
```

Une fonction statique n'a accès qu'aux champs statiques, à ses paramètres et aux fonctions statiques.

Elle n'a pas accès à l'objet courant car celui-ci n'existe pas dans son contexte (**pas de `this` dans un contexte statique**).

Jeu de piste

Un attribut statique n'existe qu'une seule fois en mémoire au contraire des variables d'instances qui existent pour chaque instance. Une fonction statique porte sur la classe en général mais sur aucune instance particulière.

```
public class C {
    private int instanceVariable;
    private static int staticField;

    public int getInstanceVariable() {
        return instanceVariable;
    }

    public static int getStaticField() {
        return staticField;
    }

    public void incrementInstanceVariable() {
        ++this.instanceVariable;
    }

    public static void incrementStaticField() {
        ++staticField;
    }
}
```

```
C c1 = new C();
C c2 = new C();

System.out.println(c1.getInstanceVariable()); //0
System.out.println(c2.getInstanceVariable()); //0

c1.incrementInstanceVariable();
System.out.println(c1.getInstanceVariable()); //1

c2.incrementInstanceVariable();
System.out.println(c2.getInstanceVariable()); //1

System.out.println(C.getStaticField()); //0

C.incrementStaticField();
System.out.println(C.getStaticField()); //1

c1.incrementStaticField(); //notation déconseillée
c2.incrementStaticField();
System.out.println(C.getStaticField()); //3
```

Déclencher une exception

Déclencher une exception

Le mot clé **throw** permet de déclencher une exception pour signaler une erreur.

```
public void setDay(int day) {  
    if(day <= 0 || day > dayInMonth()) {  
        throw new RuntimeException("error : incorrect day");  
    }  
    this.day = day;  
}
```

Pour le moment, si une `RuntimeException` est déclenchée durant l'exécution, le programme s'arrête.

Nous verrons les exceptions plus en détail dans la suite du cours.